

Skip List Coding

Coding Skip List

- The code will be completely discussed in the class. I may live code as much as I can and discuss in that process with the feedback from the students.

So, there is no explanation in this slide.

- However, the important functions are provided in the slide for you to review and writing the code on your own.

Node Class

```
class node {  
  
    public int data;  
    public node next;  
    public node prev;  
    public node up;  
    public node down;  
    public int level;  
  
    public node(int myval, int mylev) {  
        data = myval;  
        level = mylev;  
        next = null;  
        prev = null;  
        up = null;  
        down = null;  
    }  
  
}
```

Class variables

- `final public static int NEG_INF = Integer.MIN_VALUE;`
- `final public static int POS_INF = Integer.MAX_VALUE;`
-
- `public static Random rndObj = new Random();`
-
- `private ArrayList<node> levels; //store the first node of each level`
- `private int size; //keep track of how many levels do we have`

Printing the list

```
//Mainly for debugging purpose
public void printAllLevels() {

    System.out.println(levels.size()+" and "+ size);
    for (int i=0; i<size; i++) { //go to each level
        System.out.print("Level "+i+": ");
        printLevel(i);
    }
    System.out.println("-----");
}
```

```
// Prints a given level for debugging
public void printLevel(int id) {
    node cur = levels.get(id);
    while (cur != null) {
        System.out.print(cur.data+" ");
        cur = cur.next;
    }
    System.out.println();
}
```

Search Function

```
// Returns a list of nodes at each level that are right before value.
public ArrayList<node> search(int value) {

    // Store answers here.
    ArrayList<node> res = new ArrayList<node>();
    node cur = levels.get(size-1);

    // We search from top, so that we can skip terms.
    for (int i=size-1; i>=0; i--) {

        // Go down this level until we're right before an equal or bigger item.
        while (cur.next.data < value) cur = cur.next;

        // This is the floor of value on this list, so add it.
        res.add(cur);

        // Go down to the next level.
        if (i>0) cur = cur.down;
    }

    // So the list will be in the proper order, since I built it backwards.
    Collections.reverse(res);

    // Stores all of the relevant pointers.
    return res;
}
```

Building a new level with $-\text{INF}$ to $+\text{INF}$

```
// This builds level id to be an empty level.  
public node buildLevel(int id) {  
    node first = new node(NEG_INF, id);  
    node last = new node(POS_INF, id);  
    first.next = last;  
    last.prev = first;  
    return first;  
}
```

Constructor (Initialize the first level)

```
// Makes an empty list.  
public Main() {  
  
    // Initially, I am just one level with min and max.  
    levels = new ArrayList<node>();  
    levels.add(buildLevel(0));  
    size = 1;  
}
```

Connect Last (top) Level

```
// Connects the last level to the rest of the lists.  
public void connectLastLevel() {  
  
    // We can obtain both of these.  
    node top = levels.get(levels.size()-1);  
    node below = levels.get(levels.size()-2);  
  
    // Link left sides up and down.  
    top.down = below;  
    below.up = top;  
  
    // End of top list.  
    top = top.next;  
  
    // Go to end of second to top list.  
    while (below.data != POS_INF) below = below.next;  
  
    // Link right sides up and down.  
    top.down = below;  
    below.up = top;  
}
```

Insert a **key** in the list (summary)

- **Use search function:** get the list of nodes from top to the lowest level that comes right before the value we are inserting [let's call it **beforeList**]
- After this, we have the list and the first item in the list should be the item right before key in the lowest level (**I just want to insert the key after this**)
- No need to insert if they are same
- Otherwise, start a loop with $i = 0$ to size
 - Create new node with the key
 - flip the coin for 0/1 (we will intentionally make it 1 for $i=0$ as we really need to insert the key first)
 - If we are not in the lowest level, we need to update the up and down of the node (we call it curr node)
 - If you are in highest level and need to go up, create a new level and connect it with the old highest level. Add the first node of this new level to the beforeList.
 - For all cases, you will need to insert your new node after beforeList[i] position
 - Update the curr node with the new node
 - Finally, if you see that I reached size, increase the size and get out from this process

```

// Inserts value into the set, returns true iff the
// value was inserted. (False means the
// value was already in the set.
public boolean insert(int value) {
    // Find all the "previous" nodes.
    ArrayList<node> beforeList = search(value);
    // This value is already in the set.
    if (beforeList.get(0).next.data == value)
        return false;
    // Temp pointer I will use.
    node curn = null;
    int i = 0;
    // Farthest we'll go up the lists.
    while (i <= size) {
//intentionally generate 1 for first level as we need
//to at least insert it
        int val = i==0 ? 1 : rndObj.nextInt(2);
// int val = rndObj.nextInt(2);
        if (val == 0) break;
        // We've decided to create this node.
        node newn = new node(value, i);
        // Not necessary for bottom level.
        if (i > 0) {
            curn.up = newn;
            newn.down = curn;
        }
    }
}

```

```

// Special case where we are adding a new level to our list.
// We add the level and then connect it to the rest of the lists.
    if (i == size) {
        node nextL = buildLevel(size);
        levels.add(nextL);
        connectLastLevel();
        beforeList.add(nextL);
    }

    joining newn between prev and next)
    node tmpLow = beforeList.get(i);
    node tmpNext = tmpLow.next;
    newn.prev = tmpLow;
    newn.next = tmpNext;
    tmpLow.next = newn;
    tmpNext.prev = newn;

    // Need to update the object's size and get out.
    if (i == size) {
        size++;
        break;
    }
    // Go up next level.
    i++;
    curn = newn;
}
// We inserted it.
return true;
}

```

Size of the top level

```
// Returns the number of items on the top level.  
private int topLevelSize() {  
    node cur = levels.get(size-1);  
    int sz = 0;  
    while (cur != null) {  
        cur = cur.next;  
        sz++;  
    }  
    return sz;  
}
```

Delete

```
// Deletes value from the list. Returns true if value was in the list and was deleted.
// Returns false if value wasn't in the list and takes no action.
public boolean delete(int value) {

    // See if it's in the list or not.
    ArrayList<node> beforeList = search(value);
    node bottom = beforeList.get(0);
    // If not, indicate that we can't delete it.
    if (bottom.next.data != value) return false;
    // This is who we delete.
    node cur = bottom.next;
    // Go up the skip list.
    while (cur != null) {
        // Get the two to patch.
        node pNode = cur.prev;
        node nNode = cur.next;
        // Patch!
        pNode.next = nNode;
        nNode.prev = pNode;
        // Go up one level.
        cur = cur.up;
    }
    // Just in case we can trim.
    if (size > 1 && topLevelSize() == 2) {
        levels.remove(size-1);
        size--;
    }
    // Delete is completed.
    return true;
}
```

```

// Basic insert test.
public static void TestCode() {

    // Create the object.
    Main mine = new Main(); //this constructor
    creates a level as well

    // Do 100 inserts.
    for (int i=0; i<100; i++) {

        // Generate the item.
        int item = rndObj.nextInt(1000);
        System.out.println("Gen "+item);

        // Insert it.
        boolean flag = mine.insert(item);

        // Print what happened.
        if (flag)
            System.out.println("Inserted "+item);
        else
            System.out.println("Rejected "+item);

        // See all the lists.
        mine.printAllLevels();
    }
}

```

```

//Let us delete
Scanner sc = new Scanner(System.in);
int item = 0;
while(item !=-1)
{
    System.out.println("Enter an item to delete: ");
    item = sc.nextInt();
    boolean flag = mine.delete(item);
    if (flag)
    {
        System.out.println("Deleted "+item);
        mine.printAllLevels();
    }
    else
        System.out.println(item+" can not be
deleted.");
}

}

```